

ReLU-QP Integration Report

GPU-Accelerated QP Solving for Quadrotor MPC

Vrishabh Kenkre
Carnegie Mellon University, MS Mechanical Engineering
Robotics & Controls

April 2026

1. What is ReLU-QP?

ReLU-QP is a GPU-accelerated quadratic programming solver from the REx Lab at CMU (Bishop, Zhang, Gurumurthy, Tracy, Manchester — ICRA 2024) [1]. It reformulates the Alternating Direction Method of Multipliers (ADMM) as a deep neural network with ReLU activations. The key observation: every ADMM iteration consists of (a) a matrix-vector product and (b) a clamping operation — which maps exactly to a linear layer followed by a ReLU activation in a neural network.

1.1 Standard ADMM for QP

Consider the standard convex quadratic program:

$$\begin{aligned} \min_{\mathbf{y}} \quad & \frac{1}{2} \mathbf{y}^\top \mathbf{H} \mathbf{y} + \mathbf{g}^\top \mathbf{y} \\ \text{s.t.} \quad & \mathbf{c} \leq \mathbf{G} \mathbf{y} \leq \mathbf{d} \end{aligned} \tag{1}$$

where $\mathbf{H} \in \mathbb{R}^{n \times n}$ is symmetric positive definite, $\mathbf{G} \in \mathbb{R}^{m \times n}$, and $\mathbf{c}, \mathbf{d} \in \mathbb{R}^m$ define box constraints on the linear map $\mathbf{G}\mathbf{y}$.

ADMM introduces a *splitting variable* $\mathbf{z} \in \mathbb{R}^m$ and a *dual variable* (Lagrange multiplier) $\boldsymbol{\lambda} \in \mathbb{R}^m$ to decouple the cost minimization from the constraint projection. The augmented Lagrangian is:

$$\mathcal{L}_\rho(\mathbf{y}, \mathbf{z}, \boldsymbol{\lambda}) = \underbrace{\frac{1}{2} \mathbf{y}^\top \mathbf{H} \mathbf{y} + \mathbf{g}^\top \mathbf{y}}_{\text{original cost}} + \underbrace{\frac{\sigma}{2} \|\bar{\mathbf{y}} - \mathbf{y}\|^2}_{\text{regularization}} + \underbrace{\boldsymbol{\lambda}^\top (\mathbf{G}\bar{\mathbf{y}} - \mathbf{z}) + \frac{1}{2} (\mathbf{G}\bar{\mathbf{y}} - \mathbf{z})^\top \boldsymbol{\rho} (\mathbf{G}\bar{\mathbf{y}} - \mathbf{z})}_{\text{penalty for } \mathbf{G}\bar{\mathbf{y}} \neq \mathbf{z}} + \mathcal{I}_{[\mathbf{c}, \mathbf{d}]}(\mathbf{z}) \tag{2}$$

where $\sigma > 0$ is a regularization parameter, $\boldsymbol{\rho} = \text{diag}(\rho_1, \dots, \rho_m)$ is the penalty matrix, and $\mathcal{I}_{[\mathbf{c}, \mathbf{d}]}$ is the indicator function enforcing $\mathbf{c} \leq \mathbf{z} \leq \mathbf{d}$.

ADMM alternates between minimizing over $\mathbf{y}$ (unconstrained quadratic — solved in closed form) and $\mathbf{z}$ (projection onto box — clamping):

$$\boxed{\text{box}} \text{align}^+ = \underbrace{(\mathbf{H} + \sigma \mathbf{I} + \mathbf{G}^\top \boldsymbol{\rho} \mathbf{G})^{-1}}_D (-\mathbf{g} + \sigma \mathbf{y} + \mathbf{G}^\top (\boldsymbol{\rho} \mathbf{z} - \boldsymbol{\lambda})) \text{ (linear solve)}$$

$$\mathbf{z}^+ = \Pi_{[\mathbf{c}, \mathbf{d}]}(\mathbf{G}\bar{\mathbf{y}}^+ + \boldsymbol{\rho}^{-1} \boldsymbol{\lambda}) \text{ (projection / clamping)}$$

$$\boldsymbol{\lambda}^+ = \boldsymbol{\lambda} + \boldsymbol{\rho}(\mathbf{G}\bar{\mathbf{y}}^+ - \mathbf{z}^+) \text{ (dual update)}$$

Step (1.1) is the expensive step: it requires solving an $n \times n$ linear system. On CPU, this is typically done via Cholesky factorization of $\mathbf{H} + \sigma \mathbf{I} + \mathbf{G}^\top \boldsymbol{\rho} \mathbf{G}$ (OSQP uses the sparse QDLDL factorization [3]). Our C ADMM avoids this entirely by exploiting the Riccati structure of the MPC problem [2], operating on 12×12 blocks instead of the full 332×332 system.

The key insight enabling ReLU-QP: the matrix $\mathbf{D} = (\mathbf{H} + \sigma \mathbf{I} + \mathbf{G}^\top \boldsymbol{\rho} \mathbf{G})^{-1}$ can be **precomputed offline** if $\mathbf{H}$, $\mathbf{G}$, and $\boldsymbol{\rho}$ are constant across MPC steps. Only $\mathbf{g}$, $\mathbf{c}$, $\mathbf{d}$ change (they encode the current state and reference). This converts the online computation from a matrix *factorization* to a matrix-vector *product* — exactly what GPUs excel at.

1.2 The Neural Network Reformulation

ReLU-QP reorders the ADMM updates (moving the $\boldsymbol{\lambda}$ update before $\mathbf{y}$ and $\mathbf{z}$) and combines all three steps into a single matrix equation. Define the concatenated variable:

$$\mathbf{v} = \begin{bmatrix} \mathbf{y} \\ \mathbf{z} \\ \boldsymbol{\lambda} \end{bmatrix} \in \mathbb{R}^{n+2m} \quad (3)$$

Then one complete ADMM iteration is:

$$\boxed{\mathbf{v}^+ = \Pi_{[\tilde{\mathbf{c}}, \tilde{\mathbf{d}}]}(\mathbf{W} \mathbf{v} + \mathbf{b})} \quad (4)$$

where the **weight matrix** $\mathbf{W} \in \mathbb{R}^{(n+2m) \times (n+2m)}$ is:

$$\mathbf{W} = \begin{bmatrix} \mathbf{D}(\sigma \mathbf{I} - \mathbf{G}^\top \boldsymbol{\rho} \mathbf{G}) & 2 \mathbf{D} \mathbf{G}^\top \boldsymbol{\rho} & -\mathbf{D} \mathbf{G}^\top \\ \mathbf{G} \mathbf{D}(\sigma \mathbf{I} - \mathbf{G}^\top \boldsymbol{\rho} \mathbf{G}) + \mathbf{G} & 2 \mathbf{G} \mathbf{D} \mathbf{G}^\top \boldsymbol{\rho} - \mathbf{I} & -\mathbf{G} \mathbf{D} \mathbf{G}^\top + \boldsymbol{\rho}^{-1} \\ \boldsymbol{\rho} \mathbf{G} & -\boldsymbol{\rho} & \mathbf{I} \end{bmatrix} \quad (5)$$

the **bias vector** $\mathbf{b} \in \mathbb{R}^{n+2m}$ is:

$$\mathbf{b} = \begin{bmatrix} -\mathbf{D} \\ -\mathbf{G} \mathbf{D} \\ \mathbf{0} \end{bmatrix} \mathbf{g} \quad (6)$$

and the **clamping bounds** are:

$$\tilde{\mathbf{c}} = \begin{bmatrix} -\infty \\ \mathbf{c} \\ -\infty \end{bmatrix}, \quad \tilde{\mathbf{d}} = \begin{bmatrix} +\infty \\ \mathbf{d} \\ +\infty \end{bmatrix} \quad (7)$$

The $\mathbf{y}$ and $\boldsymbol{\lambda}$ blocks pass through unclamped (bounds at $\pm\infty$). Only the $\mathbf{z}$ block is clamped to $[\mathbf{c}, \mathbf{d}]$ — this is the constraint enforcement.

Why this is a neural network. The clamping operation $\Pi_{[\mathbf{c}, \mathbf{d}]}(x) = \min(\max(x, \mathbf{c}), \mathbf{d})$ can be decomposed as:

$$\Pi_{[\mathbf{c}, \mathbf{d}]}(x) = \text{ReLU}(x - \mathbf{c}) - \text{ReLU}(x - \mathbf{d}) + \mathbf{c} \quad (8)$$

where $\text{ReLU}(x) = \max(0, x)$. So each ADMM iteration (4) is a linear layer ($\mathbf{W} \mathbf{v} + \mathbf{b}$) followed by a ReLU-based activation. Running K iterations = K layers of a **weight-tied** neural network (same $\mathbf{W}$ at every layer). This is not an approximation — it is mathematically equivalent to standard ADMM.

Offline vs online. $\mathbf{W}$ depends on $\mathbf{H}$, $\mathbf{G}$, ρ (constant for time-invariant MPC). $\mathbf{b}$ depends on $\mathbf{g}$ (changes every step — encodes current state and reference). So the “weights” $\mathbf{W}$ are fixed and the “bias” $\mathbf{b}$ is updated each step via a single matrix-vector product: $\mathbf{b} = \mathbf{B}\mathbf{g}$, where $\mathbf{B} = [-\mathbf{D}; -\mathbf{G}\mathbf{D}; \mathbf{0}]$ is precomputed.

Adaptive ρ . Since $\mathbf{W}$ depends on ρ , adaptive penalty requires precomputing $\mathbf{W}$ for multiple ρ values. The implementation stores 17 $\mathbf{W}$ matrices for logarithmically-spaced $\rho \in [10^{-6}, 10^6]$ and switches between them based on the primal/dual residual ratio [3].

2. Our MPC Problem

We solve a linear MPC for a Crazyflie quadrotor in MuJoCo:

- State $\mathbf{x} \in \mathbb{R}^{12}$: position, velocity, Euler angles, angular rates
- Control $\mathbf{u} \in \mathbb{R}^4$: thrust T , torques τ_x, τ_y, τ_z
- Horizon $N = 20$, $\Delta t = 10$ ms (100 Hz), 0.2 s preview
- Cost: $Q_{\text{pos}} = 300$, $Q_{\text{vel}} = 10$, $R_T = 30$, $R_\tau = 1500$
- Bounds: $T \in [0, 0.6]$ N, $|\tau| \leq 0.0069$ N·m, $|\phi|, |\theta| \leq 35$

This QP can be formulated in two ways, with dramatically different consequences for ReLU-QP.

3. Sparse vs Condensed Formulation

3.1 Sparse (Direct) Formulation

Decision variables include states *and* controls at every timestep:

$$\mathbf{y} = [\mathbf{x}_0^\top, \mathbf{x}_1^\top, \dots, \mathbf{x}_N^\top, \mathbf{u}_0^\top, \dots, \mathbf{u}_{N-1}^\top]^\top \in \mathbb{R}^{332} \quad (9)$$

The constraint matrix $\mathbf{G}$ encodes dynamics ($\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k + \mathbf{d}$), initial condition, state bounds, and control bounds: 584 constraints total, including **240 equality constraints** for dynamics.

The ReLU-QP concatenated variable $\mathbf{v} = [\mathbf{y}; \mathbf{z}; \boldsymbol{\lambda}]$ has dimension $332 + 2 \times 584 = \mathbf{1500}$. The $\mathbf{W}$ matrix is 1500×1500 with 2.25×10^6 entries.

3.2 Condensed Formulation

Eliminate states using the prediction model:

$$\mathbf{x}_k = \mathbf{A}_d^k \mathbf{x}_0 + \sum_{j=0}^{k-1} \mathbf{A}_d^{k-1-j} \mathbf{B}_d \mathbf{u}_j + \boldsymbol{\gamma}_k \quad (10)$$

where $\boldsymbol{\gamma}_k$ accumulates gravity offsets. Define $\boldsymbol{\Gamma} \in \mathbb{R}^{(N+1)n_x \times Nn_u}$ as the matrix mapping $\mathbf{U} = [\mathbf{u}_0; \dots; \mathbf{u}_{N-1}]$ to the stacked state trajectory, and $\boldsymbol{\Phi} \in \mathbb{R}^{(N+1)n_x \times n_x}$ as the free-response matrix.

Decision variables are controls only:

$$\mathbf{y} = \mathbf{U} = [\mathbf{u}_0^\top, \mathbf{u}_1^\top, \dots, \mathbf{u}_{N-1}^\top]^\top \in \mathbb{R}^{80} \quad (11)$$

The condensed cost Hessian is:

$$\mathbf{H}_c = \boldsymbol{\Gamma}^\top \mathbf{Q}_{\text{stack}} \boldsymbol{\Gamma} + \mathbf{R}_{\text{stack}} \in \mathbb{R}^{80 \times 80} \quad (12)$$

Constraints reduce to control bounds only (80 inequality constraints, **no equalities**). The concatenated variable $\mathbf{v}$ has dimension $80 + 2 \times 80 = \mathbf{240}$. The $\mathbf{W}$ matrix is 240×240 with 57,600 entries — a **39 $\times$ reduction**.

The tradeoff: state constraints (angle bounds) are not directly enforced. They must either be included as additional inequality rows in $\mathbf{G}$ via $\mathbf{\Gamma}$, or relied upon implicitly through sufficient control authority.

4. The Sparse Formulation Failure

4.1 Symptom

ReLU-QP with the sparse formulation ($\mathbf{W} \in \mathbb{R}^{1500 \times 1500}$) failed to converge after 4000 iterations on GPU and 8000 on CPU. The adaptive ρ oscillated between 20 and 14,000 in a repeating cycle of ~ 400 iterations. Primal residual was stuck at ~ 0.103 , never crossing the convergence threshold of $\varepsilon_{\text{abs}}\sqrt{m} = 10^{-3}\sqrt{584} = 0.024$. The solver output was hover thrust with zero corrective control.

4.2 Root Cause: Condition Number Catastrophe

The $\mathbf{H}$ matrix has diagonal entries ranging from 0.1 (angular rate cost) to 10,086 (terminal cost from $\mathbf{P}_\infty$ via DARE) — a **100,000 $\times$ condition number**. Additionally, the 240 dynamics equality constraints have $\mathbf{c}_k = \mathbf{d}_k = \mathbf{d}_{\text{grav}}$, so $\mathbf{u}_k - \mathbf{l}_k = 0$. ReLU-QP internally applies $\rho_{\text{eq}} = 1000 \cdot \rho_{\text{ineq}}$ for equalities, creating two constraint populations with 1000 $\times$ different penalty weights in $\mathbf{W}$.

The combined effect: no single ρ satisfies both populations.

- High ρ (> 1000): equality constraints well-enforced (small primal residual), but the cost gradient is overwhelmed by the penalty term (dual residual explodes).
- Low ρ (< 10): cost properly optimized (small dual residual), but equalities poorly enforced (primal residual stalls).

Adaptive ρ oscillates between these regimes without converging.

4.3 Why Our C ADMM and OSQP Don't Have This Problem

OSQP uses sparse Cholesky factorization (QDLDL) to solve (1.1) directly. Direct methods compute exact solutions to the linear system regardless of conditioning.

Our C ADMM uses cached Riccati recursion (TinyMPC approach [2]). The DARE solution $\mathbf{P}_\infty$ naturally absorbs the conditioning. The backward-forward sweep operates on well-conditioned 12×12 blocks, never forming the full 1500×1500 system.

4.4 Fix Attempts on the Sparse Formulation

1. **ρ sweep** (0.01 to 100): all values hit max_iter. $\rho = 100$ came closest (primal residual 0.026 vs threshold 0.024) but never crossed.
2. **Finite bounds** (replace 10^{10} with ± 20): no effect on $\mathbf{H}$ conditioning.
3. **Variable scaling** (Ruiz-style): worsened conditioning from 10^5 to 7.5×10^5 because $\mathbf{P}_\infty$ has large off-diagonal entries.
4. **Fixed ρ** (disable adaptive): diverges because no single ρ is appropriate for the 10^8 range.

5. The Fix: Condensed Formulation

Switching to the condensed formulation eliminates states and, critically, **all 240 equality constraints**. The remaining 80 inequality constraints (control bounds) have similar ρ requirements, so adaptive ρ converges smoothly.

Metric	Sparse	Condensed
Decision variables	332	80
Constraints	584 (incl. 240 equalities)	80 (inequalities only)
$\mathbf{W}$ matrix size	1500×1500	240×240
Cold-start iterations	Diverges (> 8000)	275
Warm-start iterations	Diverges	25
CPU solve time	230 ms (divergent)	$1,047 \mu\text{s}$
Tracking SS-RMSE	500 mm (garbage)	4.9 mm

Table 1: Sparse vs. condensed formulation results for ReLU-QP on our Crazyflie MPC problem.

The ReLU-QP paper (Section IV, Figure 1 caption) confirms that all MPC benchmarks use the “preconditioned condensed formulation” — the sparse formulation is shown only as a comparison baseline for OSQP-Direct.

6. CPU vs GPU Performance

6.1 Kernel Launch Overhead

On the RTX 4070, ReLU-QP with the condensed formulation takes **$1,821 \mu\text{s}$ per solve** — actually slower than the $1,047 \mu\text{s}$ CPU time. The reason: for a 240×240 matmul (57,600 FLOPs), the GPU compute time is $\sim 1 \mu\text{s}$. But each CUDA kernel launch costs 5–10 μs of CPU-GPU synchronization overhead. With ~ 75 kernel launches per solve (25 iterations $\times \sim 3$ kernels each), the total overhead is $\sim 750 \mu\text{s}$ — dwarfing the actual computation.

Additionally, the `update()` function recomputes the bias vector $\mathbf{b}_k = \mathbf{B}_k \mathbf{g}$ for all 17 precomputed ρ values (one GPU kernel launch each), adding another $\sim 170 \mu\text{s}$ of overhead.

6.2 Where GPU Wins

Large problems. For a humanoid robot with 100+ states and 50+ controls, $\mathbf{W}$ is $\sim 1500 \times 1500$. Each matmul is 2.25×10^6 FLOPs — enough to saturate GPU compute units. The ReLU-QP paper shows order-of-magnitude speedups above ~ 50 control dimensions.

Batch solving. For RL training with domain randomization, solving 1000 QPs simultaneously with a batched ($1000 \times 240 \times 240$) tensor operation costs ~ 2 ms total. A sequential CPU solver would need $1000 \times 64 \mu\text{s} = 64$ ms.

6.3 Full Solver Comparison

7. The Bigger Picture: Solvers as Neural Networks

7.1 The MPC–RL Connection

The reformulation (4) makes explicit an architectural equivalence:

Solver	Formulation	SS-RMSE	Solve	Device	Structure
C ADMM	Sparse + Riccati	3.7 mm	64 μs	CPU	MPC-specific
OSQP	Sparse + Cholesky	4.9 mm	1,094 μ s	CPU	General sparse
ReLU-QP	Condensed + Dense	4.9 mm	1,047 μ s	CPU	General dense
ReLU-QP	Condensed + Dense	4.9 mm	1,821 μ s	GPU	General dense
ReLU-QP	Sparse + Dense	diverges	—	GPU	General dense

Table 2: All solvers on the Crazyflie figure-8 (100 Hz, tuned weights, attitude feedforward). C ADMM exploits Riccati structure for $\mathcal{O}(N \cdot n_x^2)$ per iteration vs. $\mathcal{O}((N \cdot n_u)^2)$ for condensed ReLU-QP.

$$\begin{aligned} \text{ADMM iteration:} \quad & \mathbf{v}^+ = \text{clamp}(\mathbf{W}\mathbf{v} + \mathbf{b}, \mathbf{c}, \mathbf{d}) \\ \text{Neural network layer:} \quad & \mathbf{h}^+ = \text{ReLU}(\mathbf{W}\mathbf{h} + \mathbf{b}) \end{aligned}$$

An MPC controller running K ADMM iterations is a K -layer neural network where the weights $\mathbf{W}$ are derived from physics ($\mathbf{A}_d, \mathbf{B}_d, \mathbf{Q}, \mathbf{R}$). An RL policy is also a K -layer neural network where the weights are learned from data via backpropagation. The computation is identical; only the weight source differs:

- **MPC:** weights from dynamics and cost $\rightarrow$ optimal, interpretable, constraint-respecting
- **RL policy:** weights from training data $\rightarrow$ fast, robust to model error, no online optimization
- **Dagger distillation:** MPC generates expert data $\rightarrow$ RL network learns to imitate $\rightarrow$ deploys cheap network

7.2 State of the Art (2024–2025)

1. **TinyMPC** (ICRA 2024 Best Paper): Cached Riccati ADMM for embedded CPU. Our C ADMM implements this.
2. **ReLU-QP** (ICRA 2024): ADMM as neural network for GPU. Enables PyTorch/JAX integration.
3. **DiffMPC** (2025, Adabag et al.): Full nonlinear MPC on GPU with SQP + PCG. Differentiable — enables RL *through* the solver. Demonstrated Toyota Supra drifting through water puddles.
4. **FastDOC** (2025): Faster backward pass through differentiable MPC via Gauss-Newton structure exploitation.

The trajectory is clear: MPC solvers are becoming neural networks. The boundary between optimization-based and learning-based control is dissolving.

8. Key Takeaways

1. **Formulation matters more than hardware.** Switching from sparse (332 vars) to condensed (80 vars) turned ReLU-QP from divergent to convergent. No amount of GPU power fixes a 10^5 condition number.
2. **Structure-exploiting beats general-purpose for small problems.** C ADMM (Riccati) is $16\times$ faster than ReLU-QP because it reduces 1500-dimensional dense operations to sequences of 12-dimensional structured operations.

3. **GPU needs large problems.** ReLU-QP’s advantage appears above ~ 50 control dimensions, where the dense matmul saturates GPU compute and kernel launch overhead becomes negligible. For $n_u = 4$, the overhead dominates.
4. **Equality constraints are poison for dense ADMM.** The sparse formulation’s 240 dynamics equalities, with their $1000\times$ higher ρ , create a condition number catastrophe that no adaptive ρ strategy can resolve in the dense formulation.
5. **The MPC-RL connection is architecturally explicit.** ReLU-QP demonstrates that “solving a QP” and “running a forward pass” are the same computation. This enables a unified pipeline: physics-derived weights for MPC, learned weights for RL, distillation between them.

References

- [1] A. L. Bishop, J. Z. Zhang, S. Gurumurthy, K. Tracy, and Z. Manchester, “ReLU-QP: A GPU-accelerated quadratic programming solver for model-predictive control,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, pp. 13285–13292.
- [2] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, “TinyMPC: Model-predictive control on resource-constrained microcontrollers,” in *Proc. IEEE ICRA*, 2024. Best Paper in Automation.
- [3] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.
- [4] E. Adabag, M. Greiff, J. Subosits, and T. Lew, “Differentiable model predictive control on the GPU,” *arXiv:2510.06179*, 2025.
- [5] B. Amos, I. Jimenez, J. Sacks, B. Boots, and J. Z. Kolter, “Differentiable MPC for end-to-end planning and control,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.